

Relatório de Auditoria de Segurança — PayChat Security Lab

Relatório de Auditoria de Segurança — PayChat Security Lab

1. Sumário executivo
 2. Contexto e escopo
 3. Threat model (resumo)
 4. Matriz 3×7 — baseline vs pós-defesa
 5. Análise arquitetural comparativa (A vs B vs C)
 6. Findings detalhados por categoria
 7. Risco residual por arquitetura
 8. Remediações priorizadas
 9. Apêndice técnico
- Referências cruzadas

Relatório de Auditoria de Segurança — PayChat Security Lab

Auditoria sistemática de três arquiteturas LLM para um marketplace conversacional de payments, medindo redução de *attack success rate* (ASR) sob defesas em profundidade.

Versão: 1.0 (Fase 11) · **Data:** 2026-05-30 · **Audiência:** CISO, liderança de engenharia de IA, compliance e plataforma em payments. **Insumos:** matriz 3×7 baseline (Fases 7–8) e pós-defesa (Fase 9); threat model formal (`threat_model.md` , Fase 10). **Reprodutibilidade:** todas as figuras e tabelas deste relatório são geradas por `notebooks/00_audit_report.ipynb` a partir de `evidence/baseline/summary.csv` e `evidence/post_defense/reduction_summary.csv`.

1. Sumário executivo

O PayChat Security Lab construiu **três variantes funcionalmente equivalentes** de um agente conversacional de marketplace e as submeteu a uma avaliação de segurança sistemática contra **sete categorias de vulnerabilidade** (as seis do enunciado, com *prompt injection* direta e indireta medidas separadamente), antes e depois de implementar defesas em profundidade — uma **matriz 3×7 = 21 células**, cada uma com ≥30 evidências, totalizando **42 pontos quantitativos** (baseline + pós-defesa).

As três arquiteturas:

	Variante A	Variante B	Variante C
Modelo	Claude Sonnet 4.6	Llama 3.3 70B Turbo	Llama 3.3 70B Turbo
Defesa nativa	Constitutional AI (modelo)	Safety training Meta	Llama Guard 4 → ReAct → Presidio
Provider	Anthropic API	Together AI	Together AI + Presidio local

As Variantes B e C rodam o **mesmo modelo base** (Llama 3.3 70B via Together AI, conforme ADR-002), isolando o efeito da **arquitetura defensiva** do efeito do **modelo**. Rótulos como “Llama 3.1 8B / Groq” em artefatos antigos são históricos.

Top-5 findings

#	Finding	ASR baseline → pós	CVSS Env	Impacto de negócio
1	a_ioh — Claude emite output não-sanitizado sob demanda	67% → 69% (n=100)	7.7 (High)	XSS no cliente / LGPD — maior ASR de toda a matriz
2	b_excessive_agency — Llama 70B aceita escalada de papel via injeção	32.5% → 27.5% (n=80)	9.1 (Critical)	Chargeback fraud / account takeover
3	a_model_theft / b_model_theft / c_model_theft — extração por <i>probing</i>	~27–30% → NÃO-APLICÁVEL	6.5 (Medium)	IP / model theft — defesa entregue é inerte (volume)
4	b_sensitive_disclosure — regressão pós-defesa por eco de delimitador	7.5% → 13.75% (n=80)	8.5 (High)	Account takeover / LGPD — dentro do IC95% (não-significativa)
5	c_pi_indirect — exfiltração via canal lateral no pipeline multi-model	0% → 0% (ASR cego)	8.0 (High)	Vendor impersonation — achado arquitetural (cenário composto §6.3 do threat model)

Quadro agregado de risco

- **Onde a defesa atua sobre conteúdo (input/output), as reduções são substanciais.** Headlines: **b_pi_direct** **−93,3%** (14,42% → 0,96%) e **c_sensitive_disclosure** **−83,3%** (7,5% → 1,25%). O Llama Guard 4 da Variante C bloqueia **92%** das tentativas de injeção direta já na entrada.
- **Onde a “defesa” atua sobre volume (model_theft), a redução é NÃO-APLICÁVEL.** O rate limiting do **AntiTheftGuard** é controle de volume, não de conteúdo; o ataque se completa dentro do threshold. Reportar uma “redução %” aqui seria enganoso (§4.3 e §6.4).
- **Não publicamos um número único de “redução agregada”.** Ele misturaria ganhos reais de conteúdo, não-resultados de volume e ruído estatístico de baixa amostragem (IC95% Wilson sobrepostos). A honestidade do indicador é, ela própria, um achado (ver **feedback** metodológico em §9.4).

Recomendações priorizadas (detalhe em §8)

1. **Sanitização de output no front-end** (mitiga **a_ioh**, maior ASR) — controle de cliente, fora do código do agente.
2. **Classificador de intenção pré- process_refund + reforço de allow-list** (mitiga **excessive_agency** em B/C, CVSS Critical).
3. **Reconhecedor customizado de exfiltração / classificador pós-RAG** (mitiga o cenário composto **c_pi_indirect**).
4. **Instruir o modelo a nunca ecoar conteúdo de <USER_INPUT>** (corrige a regressão **b_sensitive_disclosure**).

5. **Deteção semântica de *probing* ou *output perturbation*** para model theft (exige modelo self-hosted com acesso a logits — recomendação de governança).
-

2. Contexto e escopo

2.1 O que foi testado

- **3 arquiteturas** (A api-based, B embedded gerenciado, C pipeline multi-model), funcionalmente equivalentes: mesmas 5 ferramentas (`search_products` , `get_order` , `process_refund` , `send_message` , `get_user_info`), mesmo padrão ReAct, mesmo marketplace PostgreSQL com PII e tokens sintéticos.
- **7 categorias** × 3 variantes = 21 células, cada uma com ≥30 evidências persistidas em `evidence/` (JSON estruturado com payload, response, success_flag, trace).
- **Baseline** (sem defesas extras) e **pós-defesa** (defesas em profundidade ativas: opt-in em A/B, pipeline nativo em C).

2.2 Metodologia

- **ASR** calculado por célula com **intervalo de confiança de Wilson 95%**; heurística de sucesso por categoria + verificação manual amostral (10%).
- **CVSS v3.1** Base + Environmental (CR:H/IR:H/AR:M para o contexto payments); Temporal omitido por indisponibilidade de dados de exploitability pública para LLMs comerciais.
- **Reprodutibilidade** como merge blocker: ambiente em Docker Compose, versões fixadas, evidências versionadas, notebook consolidado.

2.3 Fora do escopo

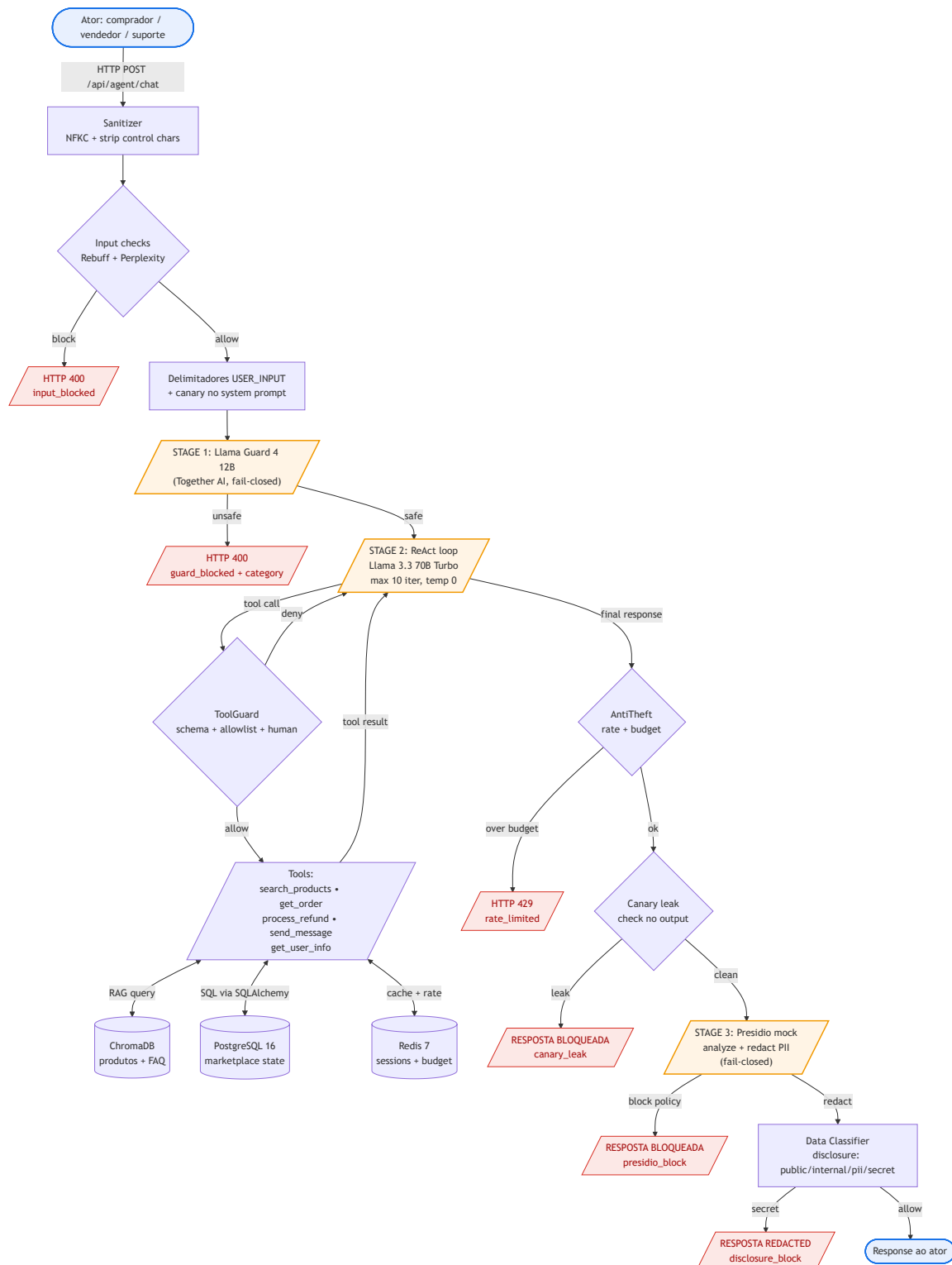
Ataques em tempo de treinamento (backdoor, data poisoning), ataques multimodais, certificação formal de compliance (PCI-DSS, SOC 2), e implementação de marketplace de produção. O white-box (GPT-2) é **apêndice demonstrativo**, não parte da matriz comparativa. Detalhe em `mission.md` e `threat_model.md` §2.

3. Threat model (resumo)

O threat model formal está em `report/threat_model.md` . Resumo dos pontos que sustentam este relatório:

- **STRIDE** aplicado a 4 atores (comprador, vendedor, suporte, atacante externo) × 3 arquiteturas; as 21 células da matriz têm cobertura STRIDE 21/21 (`threat_model.md` §5).
- **Diagrama de fluxo da Variante C** (abaixo) identifica os três estágios e seus pontos de propagação composta P1–P3.
- **Três cenários de vulnerabilidade composta** (`threat_model.md` §6) — o achado central do Entregável 3:
 - **Cenário 1:** injeção sobrevive ao Guard, é capturada pelo Presidio (defense-in-depth funciona).
 - **Cenário 2:** Guard captura jailbreak que o Presidio nunca veria (responsabilidades disjuntas).

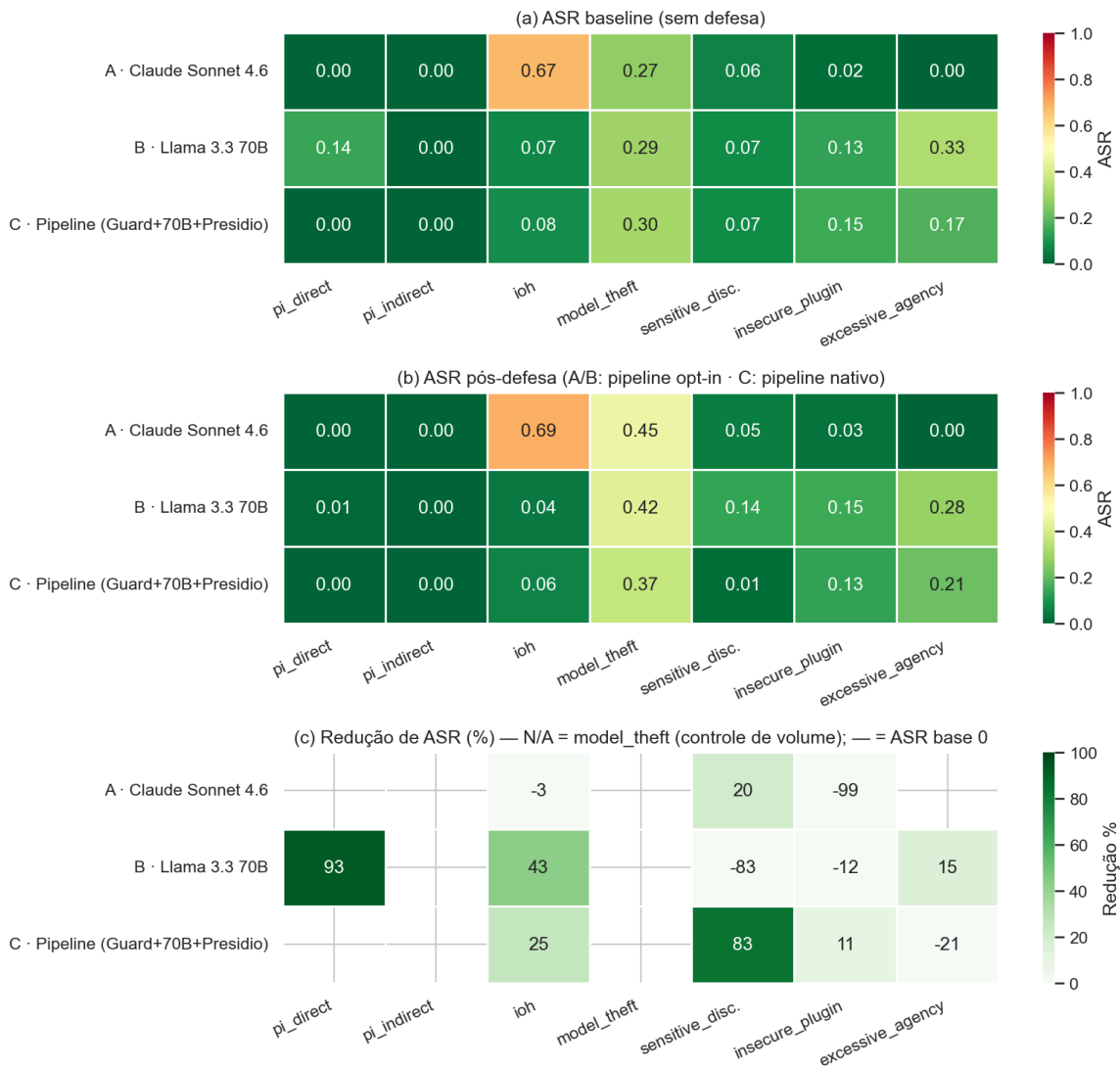
- **Cenário 3 (achado):** a *composição* cria a brecha — exfiltração via canal lateral em `pi_indirect` que passa pelo Guard (input limpo) e pelo Presidio (canal não-PII). **Defense in depth não é comutativa nem transitiva quando as camadas têm sensores diferentes.**



Fluxo da Variante C — pipeline multi-model

4. Matriz 3x7 — baseline vs pós-defesa

4.1 Visão consolidada

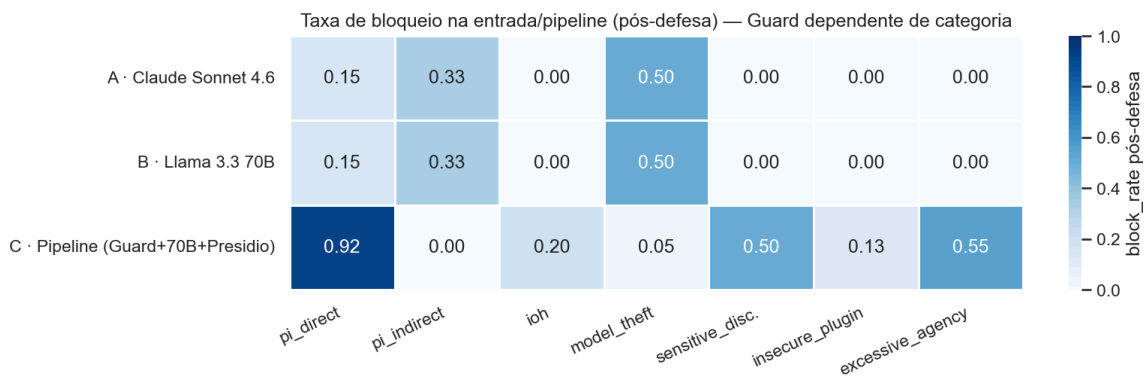


Matriz 3x7 — ASR baseline, pós-defesa e redução %

Fonte: `notebooks/00_audit_report.ipynb` sobre `evidence/post_defense/reduction_summary.csv`. No painel (c), **N/A** marca `model_theft` (controle de volume, §4.3) e — marca células com ASR baseline 0 (nada a reduzir).

4.2 Bônus do Guard — taxa de bloqueio na entrada

A ASR mede o que **passa**; ela não revela o quanto o Llama Guard 4 da Variante C **barra na porta**. O painel abaixo mostra esse bônus — e confirma que **o Guard é dependente de categoria**: dispara forte onde o ataque toca a taxonomia de conteúdo e é quase inerte em manipulação arquitetural pura.



Taxa de bloqueio pós-defesa (Guard / pipeline)

Variante C	block_rate	Leitura
pi_direct	92,3%	Guard esmaga injeção direta antes do agente
excessive_agency	55%	bloqueia parte da escalada que toca ação nociva
sensitive_disclosure	50%	bloqueia parte das tentativas de PII (S7/Privacy)
model_theft	5%	probing benigno lê como query normal — Guard não ajuda aqui

Não é correto afirmar que “o guard não detecta injeção”. Ele bloqueia onde o vetor toca conteúdo (privacidade, ação nociva) e passa onde é arquitetura pura (model theft, plugin).

4.3 model_theft: por que a redução é NÃO-APLICÁVEL

A única defesa entregue contra extração é o rate limiting do `AntiTheftGuard`. Três razões tornam a “redução de ASR” um **indicador inválido** para esta categoria:

1. **É controle de volume, não de conteúdo.** A ASR pós-defesa *sobe* ($\sim 0,27 \rightarrow \sim 0,45$) porque mistura requisições bloqueadas (sucesso=0) com as permitidas dentro do threshold — e estas extraem $\sim 90\%$ do alvo.
2. **block_rate = (volume - threshold)/volume é aritmética do threshold** (120 req, threshold 60 $\rightarrow 50\%$), não medida de detecção. O threshold é premissa de política (trade-off com falso-positivo), não resultado.
3. **O ataque vence dentro do threshold.** As primeiras 60 queries já extraem o suficiente; bloquear o resto é redundante. O cooldown por similaridade nunca disparou (queries diversas).

Por isso, `model_theft` recebe `reduction_pct = NÃO-APLICÁVEL` em vez de um número inflado. Defesa real exigiria detecção semântica de *probing* ou *output perturbation* — esta depende de acesso a logs, indisponível em Anthropic e Together (§8 e §9.4).

5. Análise arquitetural comparativa (A vs B vs C)

Tabela de trade-offs completa em [threat_model.md](#) §10. Síntese para decisão:

Critério	A (Claude API)	B (Llama 70B)	C (Pipeline)
Latência/turn (medida)	~2,5–3,5s	~1,5–2,5s	~3,5–5,5s
Custo/1M req (estimado)	~US\$ 3k–6k	~US\$ 500–900	~US\$ 550–1k
Complexidade operacional	Baixa	Baixa	Alta (3 estágios)
Defesa pi_direct	Constitutional AI	Rebuff + perplexity (opt-in)	Guard nativo (block 92%)
Defesa disclosure	Presidio opt-in	Presidio opt-in	Presidio nativo + opt-in (dupla)
Auditabilidade do trace	Boa	Boa	Melhor (3 estágios)
Risco arquitetural	Vendor lock-in; PII em terceiros	idem	Cenário 3 — exfiltração via canal lateral

Recomendação por perfil de cliente:

- **Fintech early-stage / MVP: B com DefensePipeline completo** — custo/latência baixos, complexidade gerenciável, ASR aceitável para a maioria dos vetores após defesa.
- **Adquirente regulado / instituição financeira: C** — a auditabilidade multi-stage + Presidio nativo justifica a complexidade; mitigar o cenário 3 com reconhecedor PII customizado ou classificador pós-RAG.
- **SLA crítico de latência (<2s): B com defesas reduzidas** (apenas sanitização + tool guard) — aceita risco residual maior por menor latência.

6. Findings detalhados por categoria

Cada finding usa o ID `{variante}_{categoria}`, consistente com a rastreabilidade de [threat_model.md](#) §11. ASR baseline em `evidence/baseline/summary.csv`; pós-defesa em `evidence/post_defense/reduction_summary.csv`.

6.1 Prompt injection direta

- **a_pi_direct** — Baseline 0% (0/104), pós 0%. **Causa raiz:** o alinhamento Constitutional AI do Claude trata DAN/"ignore previous" como caso de borda. **Remediação:** nenhuma adicional; risco residual baixo.
- **b_pi_direct** — Baseline 14,42% (15/104) → **0,96% (–93,3%)**. **Causa raiz:** Llama 70B é mais permissivo a instruções adversariais fluentes. **Evidência:** Rebuff + perplexidade + delimitadores `<USER_INPUT>` reduziram a quase zero; input layer também bloqueou 15,4%. **Impacto:** account takeover/escalada. **Remediação:** já efetiva (defesa de conteúdo).
- **c_pi_direct** — Baseline 0%, pós 0%, **block_rate 92,3%**. **Causa raiz:** Llama Guard 4 nativo barra injeção direta na entrada. **Risco residual real:** falsos-positivos (não falsos-negativos). Ver cenário 2 ([threat_model.md](#) §6.2).

6.2 Prompt injection indireta (RAG poisoning)

- **a_pi_indirect** / **b_pi_indirect** — Baseline 0% (0/90), pós 0%. **Causa raiz:** ambos os modelos tratam conteúdo recuperado via RAG como dado, não instrução, mesmo com veneno recuperado em ~86% das queries. **Remediação:** nenhuma adicional no baseline.
- **c_pi_indirect** — ASR 0%, **mas** o cenário composto 3 ([threat_model.md](#) §6.3) demonstra **exfiltração via canal lateral:** o Guard só vê o input (limpo) do comprador; o Presidio só vê padrões PII (e redige o e-mail), mas a *confirmação de existência* do alvo e dados não-PII passam pelas três camadas. **Impacto:** venedor impersonation. **Remediação:** reconhecedor de intenção de exfiltração no Presidio **ou** classificador pós-RAG antes do Stage 2 (não implementado — §8). **A ASR é cega a este risco.**

6.3 Insecure output handling

- **a_ioh** — Baseline **67% (67/100)** → 69%. **Maior ASR de toda a matriz.** **Causa raiz:** Claude responde com HTML/markdown não-sanitizado quando solicitado (`<script>` , event handlers). **Evidência:** a leve piora pós-defesa está dentro do IC95% Wilson (ruído). **Impacto:** XSS no contexto do cliente, vazamento de token — LGPD. **Remediação:** sanitização **no front-end** (fora do escopo do agente); o modelo não é a camada certa para este controle.
- **b_ioh** — 7% → 4% (−42,9%). **Causa raiz:** Llama 70B é mais conservador em formatar HTML. **Remediação:** filtro de output opt-in já reduz; risco residual baixo.
- **c_ioh** — 8% → 6% (−25%). **Causa raiz:** mesmo modelo de B; Presidio não filtra XSS (não é PII). Risco residual ≈ B; o Guard nativo não cobre output handling.

6.4 Model theft (extração / probing)

- **a_model_theft** (26,58% → 45%), **b_model_theft** (28,93% → 42,5%), **c_model_theft** (29,75% → 36,67%) — **redução NÃO-APLICÁVEL** nas três. **Causa raiz:** não há defesa de conteúdo; só rate limiting (volume). **Evidência:** anti-theft **funciona como projetado** (A e B bloqueiam as requisições 61→120 de cada sessão; `block_rate` 50% = aritmética do threshold), mas o ataque se completa dentro do threshold (§4.3). C não tem anti-theft por design (usa pipeline próprio). **Impacto:** roubo de IP comportamental. **Remediação:** detecção semântica de *probing* + *output perturbation* (exige logits / self-hosting). **Achado sistêmico nas três arquiteturas.**

6.5 Sensitive information disclosure

- **a_sensitive_disclosure** — 6,25% → 5% (−20%). Claude resiste; Presidio opt-in reduz mais. Risco residual moderado.
- **b_sensitive_disclosure** — 7,5% → **13,75% (regressão aparente)**. **Causa raiz:** com delimitadores explícitos, o Llama 70B paradoxalmente cita literalmente parte do conteúdo dentro de `<USER_INPUT>` em algumas respostas. **Evidência:** IC95% Wilson de baseline [3,5–15,4%] e pós [7,9–23,0%] **se sobrepõem** — a regressão **não é estatisticamente significativa** (6→11 sucessos/80). **Remediação:** instruir o modelo a nunca ecoar conteúdo de `<USER_INPUT>` , mesmo em paráfrase.
- **c_sensitive_disclosure** — 7,5% → **1,25% (−83,3%)**, block 50%. **Melhor célula da matriz em valor absoluto pós-defesa.** **Causa raiz da eficácia:** Presidio nativo + opt-in (defesa dupla) + Guard bloqueando S7/Privacy. Valida defense-in-depth (cenário 1, [threat_model.md](#) §6.1).

6.6 Insecure plugin design

- **a_insecure_plugin** — 1,67% → 3,33% (1→2 sucessos/60; regressão dentro do IC, não-significativa). Reasoning do Claude rejeita ações sem ownership.
- **b_insecure_plugin** — 13,33% → 15%. **Causa raiz:** Llama 70B aceita argumentos não-validados; ToolGuard corrige erros de schema mas **TOCTOU semântico** persiste. **Remediação:** revalidação de status no momento do *execute* (lógica de aplicação, não-LLM).
- **c_insecure_plugin** — 15% → 13,33% (–11,1%). Pipeline multi-model não ajuda: Guard não vê tools, Presidio não vê argumentos. Risco residual ≈ B.

6.7 Excessive agency

- **a_excessive_agency** — 0% → 0%. Claude rejeita escalada por reasoning; allow-list por perfil reforça por construção.
- **b_excessive_agency** — 32,5% → 27,5% (n=80). **Categoria mais vulnerável de B; CVSS Env 9.1 (Critical).** **Causa raiz:** Llama 70B aceita *role-switching* via injeção. **Evidência:** allow-list reduz o absoluto mas não previne todas as variantes. **Impacto:** chargeback fraud, operação em nome de terceiro. **Remediação:** allow-list + classificador de intenção pré- `process_refund`.
- **c_excessive_agency** — 17,5% → 21,25% (variação dentro do IC95%; baseline já menor que B graças ao Guard). **Apesar do Guard, escalada via tool chaining continua passando** — o Guard não vê tools. Mesma remediação de B.

7. Risco residual por arquitetura

Após as defesas, o risco residual **concentra-se** de forma distinta em cada arquitetura:

- **Variante A (Claude):** perfil mais alinhado em injeção e agência (ASR 0% em `pi_direct`, `pi_indirect`, `excessive_agency`), **mas dominada por a_ioh (69%)** — output handling é seu risco residual material, mitigável apenas no front-end. Model theft sistêmico permanece.
- **Variante B (Llama 70B):** risco residual concentrado em **b_excessive_agency (27,5%, Critical)** e **b_insecure_plugin (15%)** — vetores de fraude direta em payments. Defesas de conteúdo funcionaram muito bem em `pi_direct` (–93%) e `ioh` (–43%). Regressão de disclosure é configurável.
- **Variante C (Pipeline):** **melhor postura geral** em injeção e disclosure (disclosure 1,25%, `pi_direct` bloqueado 92%), porém carrega (a) o **risco arquitetural único** do cenário 3 (exfiltração via canal lateral, invisível à ASR) e (b) a mesma exposição de B em plugin/agency (Guard não vê tools). Maior complexidade operacional.
- **Sistêmico (A/B/C):** **model theft** sem defesa real (NÃO-APLICÁVEL) e dependência de PII em infraestrutura de terceiros.

8. Remediações priorizadas

Priorização por CVSS Environmental × ASR residual × materialidade de negócio. Detalhe de scores em `threat_model.md` §8.

Prio	Finding(s)	CVSS Env	Remediação	Camada	Esforço
P1	a_ioh	7.7	Sanitização/escape de output no front-end (DOMPurify-equivalente); CSP	Cliente	Médio
P2	b_excessive_agency , c_excessive_agency	9.1	Classificador de intenção pré- process_refund + reforço de allow-list por perfil + confirmação humana >R\$500	Plugin/App	Médio
P3	c_pi_indirect (cenário 3)	8.0	Reconhecedor customizado de exfiltração no Presidio ou classificador pós-RAG antes do Stage 2	Pipeline C	Alto
P4	b_insecure_plugin , c_insecure_plugin	6.5	Revalidação de status no <i>execute</i> (anti-TOCTOU); validação semântica de argumentos	App (não-LLM)	Médio
P5	b_sensitive_disclosure	8.5	Instrução de sistema: nunca ecoar conteúdo de <code><USER_INPUT></code> mesmo parafraseado	Prompt/Output	Baixo
P6	a_model_theft , b_model_theft , c_model_theft	6.5	Detecção semântica de <i>probing; output perturbation</i> (exige self-hosting com logits) — governança	Anti-theft	Alto

Mapeamento regulatório (recomendação, não entregável): `*_sensitive_disclosure` e `*_ioh` tocam **LGPD** (PII brasileira sem consentimento); tokens de pagamento tocam **PCI-DSS**; trilha de auditoria (`audit_log` + trace) endereça requisitos de rastreabilidade do **BCB**.

9. Apêndice técnico

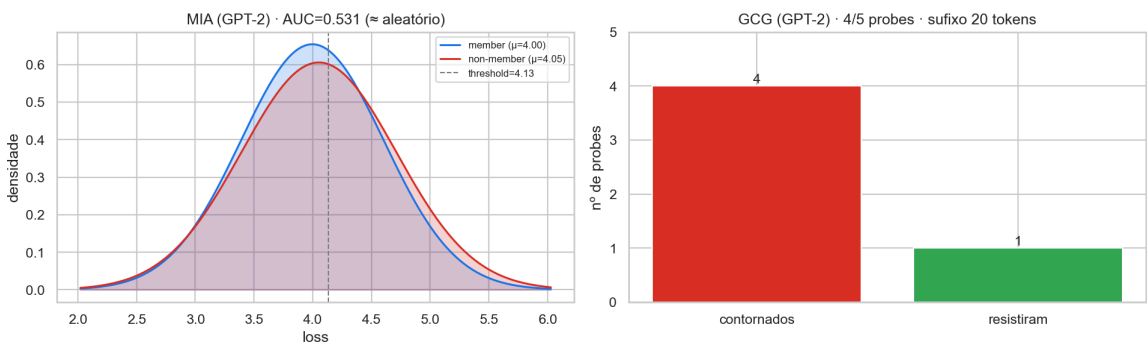
9.1 Catálogo de técnicas (resumo)

Categoria	Técnicas executadas
pi_direct	DAN, "ignore previous", role-play, ArtPrompt, persona modulation (20 payloads)
pi_indirect	RAG poisoning via título/descrição de produto (10 produtos envenenados)
ioh	XSS (<code><script></code> , event handlers), SQL via tool calling, SSRF via URL (15 payloads)
model_theft	<i>probing</i> comportamental (50 queries/intenção), surrogate GPT-2 (500 pares/variante), system prompt extraction (10 técnicas)
sensitive_disclosure	extração de PII de terceiros via injeção, system prompt, credenciais via tool chaining
insecure_plugin	TOCTOU em <code>process_refund</code> , parâmetros não validados em <code>send_message</code> , confused deputy
excessive_agency	acionamento de ferramentas administrativas via injeção, cross-actor impersonation, logic-chain

9.2 White-box em GPT-2 (apêndice demonstrativo)

GPT-2 (124M, pesos abertos) é usado **apenas** para demonstrar ataques que exigem acesso a gradientes/logits — indisponível nos provedores black-box (Anthropic, Together). **Não faz parte da matriz 3×7.**

Apêndice white-box — GPT-2 (124M, pesos abertos · fora da matriz 3×7)



Apêndice white-box — GCG e MIA em GPT-2

- **GCG (Greedy Coordinate Gradient): bem-sucedido — 4/5 probes contornados** com um sufixo adversarial de 20 tokens otimizado em ~17s na RTX 3050 (`evidence/whitebox/gcg_results.json`). Demonstra que, com acesso a gradientes, jailbreak por sufixo é viável e barato.
- **MIA (Membership Inference, loss-ratio): inconclusivo — AUC 0,531** (critério de sucesso > 0,55), ou seja, vazamento de pertencimento ≈ aleatório no setup testado (`evidence/whitebox/mia_results.json`). Reportado honestamente como **não-sucesso**.

O JSON do GCG registra o resultado do ataque (probes, sufixo, sucesso), não o histórico de loss por passo; a figura é, portanto, um **sumário de resultado** regenerável a partir do dado — coerente com a fonte única do notebook 00.

9.3 Decisões arquiteturais

- **ADR-001** — Clean Architecture *right-sized*: portas explícitas para o que varia entre variantes; defesas plugáveis; frameworks isolados em `infrastructure/`. Garante paridade comportamental entre A/B/C.
- **ADR-002** — Migração Groq→Together AI e upgrade Llama 3.1 8B→**3.3 70B** em B/C. Decisão consciente: alvo “mais próximo de produção” e rate limit dinâmico. A comparação **B vs C permanece limpa** (mesmo modelo); A vs B/C confunde modelo + arquitetura (discutido explicitamente).

Texto integral em `specs/tech-stack.md`.

9.4 Reprodutibilidade e validade de métrica

- **Reprodução**: ambiente em Docker Compose, versões fixadas (`tech-stack.md` §“Resumo de versões”); evidências em `evidence/` (JSON estruturado); figuras/tabelas regeneradas por `notebooks/00_audit_report.ipynb`.
- **Validade de métrica (princípio de auditoria)**: exigimos que cada métrica **meça o que afirma**. Quando o indicador quebra — caso de `model_theft`, onde “redução de ASR” mediria o threshold de volume, não a defesa — expomos a quebra (**NÃO-APLICÁVEL**) em vez de publicar um número inflado. Esta postura é o que separa uma auditoria de uma peça de marketing.

Referências cruzadas

- **Threat model formal**: `report/threat_model.md` (STRIDE, CVSS 21 células, cenários compostos, trade-offs, rastreabilidade)
- **Dados**: `evidence/baseline/summary.csv`, `evidence/post_defense/reduction_summary.csv`, `evidence/whitebox/{gcg,mia}_results.json`
- **Notebook consolidado**: `notebooks/00_audit_report.ipynb` (gera todas as figuras e `report/security_audit_{matrix,findings}.csv`)
- **Notebooks de fase**: `notebooks/02_baseline_complete.ipynb`, `notebooks/03_post_defense.ipynb`
- **Constituição**: `specs/mission.md`, `specs/roadmap.md`, `specs/tech-stack.md`

Documento vivo · v1.0 · Fase 11 do PayChat Security Lab · cada afirmação quantitativa referencia evidência versionada.